

Guía práctica: Pruebas parametrizadas con pytest

Este material introduce las pruebas parametrizadas con `pytest` mediante pequeñas actividades no evaluables. El objetivo es practicar estos conceptos para aplicarlos después en una actividad evaluable.

1. Introducción rápida

Las pruebas parametrizadas permiten ejecutar la misma función de prueba con distintos datos de entrada sin duplicar código. Para ello se usa el decorador `@pytest.mark.parametrize`. [Documentación oficial pytest][1]

```
import pytest

@pytest.mark.parametrize("entrada, esperado", [
    (1, 2),
    (3, 4),
])
def test_ejemplo(entrada, esperado):
    assert entrada + 1 == esperado
```

Cada tupla de la lista hace que se ejecute la prueba una vez con esos valores. [1][2]

2. Actividad 1 – Primera prueba parametrizada

Objetivo

Crear tu primera prueba parametrizada probando una función sencilla de suma.

Enunciado

Partiendo de la siguiente función `suma(a, b)`, escribe una prueba con `@pytest.mark.parametrize` que compruebe varios casos:

- $2 + 3 = 5$
- $-1 + 1 = 0$
- $0 + 0 = 0$

archivo: calculadora.py

```
def suma(a, b):  
    return a + b
```

Crea el archivo `test_calculadora.py` y escribe la prueba parametrizada.

Pistas

- Importa `pytest` y la función `suma`.
 - Usa una lista de tuplas con los valores (`a`, `b`, `resultado_esperado`).
-

Solucionario (Actividad 1)

```
# archivo: test_calculadora.py  
import pytest  
from calculadora import suma  
  
@pytest.mark.parametrize("a,b,esperado", [  
    (2, 3, 5),  
    (-1, 1, 0),  
    (0, 0, 0),  
)  
def test_suma(a, b, esperado):  
    assert suma(a, b) == esperado
```

La cadena `"a,b,esperado"` define los nombres de los parámetros, y cada tupla de la lista se asigna en orden a esos nombres. [1][2]

3. Actividad 2 – Casos con errores esperados

Objetivo

Usar parametrización en combinación con excepciones esperadas.

Enunciado

Trabajaremos con una función de división que lanza una excepción al dividir entre 0.

```
# archivo: calculadora.py
```

```
def division(a, b):  
    return a / b
```

Crea una prueba parametrizada que compruebe estos casos:

- $10 / 2 = 5$
- $3 / 1 = 3$
- División por cero: $5 / 0$ debe lanzar `ZeroDivisionError`

Para el caso de división por cero, utiliza `pytest.raises` dentro del test. [2][3]

Pistas

- Un enfoque sencillo es separar los casos "normales" del caso que lanza excepción.
 - También puedes usar un campo adicional en los parámetros para indicar si esperas excepción.
-

Solucionario (Actividad 2)

Opción A: casos separados

```
# archivo: test_division.py  
import pytest  
from calculadora import division  
  
@pytest.mark.parametrize("a,b,esperado", [  
    (10, 2, 5),  
    (3, 1, 3),
```

```

])
def test_division_ok(a, b, esperado):
    assert division(a, b) == esperado

def test_division_por_cero():
    with pytest.raises(ZeroDivisionError):
        division(5, 0)

```

Opción B: un solo test parametrizado con "modo"

```

# archivo: test_division_param.py
import pytest
from calculadora import division

@pytest.mark.parametrize("a,b,esperado,espera_error", [
    (10, 2, 5, False),
    (3, 1, 3, False),
    (5, 0, None, True),
])
def test_division(a, b, esperado, espera_error):
    if espera_error:
        with pytest.raises(ZeroDivisionError):
            division(a, b)
    else:
        assert division(a, b) == esperado

```

Este patrón mezcla casos con resultado normal y casos que lanzan excepción en una misma función de prueba. [2][3]

4. Actividad 3 – Parametrización a nivel de clase

Objetivo

Aplicar `@pytest.mark.parametrize` a una clase de tests para reutilizar los mismos datos en varios métodos.

Enunciado

Imagina una pequeña "calculadora" que de momento solo suma y resta.

```
# archivo: calculadora.py
```

```
def suma(a, b):  
    return a + b
```

```
def resta(a, b):  
    return a - b
```

Crea una clase de pruebas TestCalculadora con:

- test_suma que use los parámetros a, b, esperado_suma
- test_resta que use a, b, esperado_resta

Aplica la parametrización en la clase para que ambos métodos se ejecuten con los mismos datos. [1]

Pistas

- Puedes parametrizar con `@pytest.mark.parametrize` encima de la clase.
 - Reutiliza los parámetros a y b en ambos métodos.
-

Solucionario (Actividad 3)

```
# archivo: test_calculadora_clase.py  
import pytest  
from calculadora import suma, resta
```

```
@pytest.mark.parametrize("a,b,esperado_suma,esperado_resta",  
    (5, 2, 7, 3),  
    (10, 5, 15, 5),  
])
```

```
class TestCalculadora:
```

```
    def test_suma(self, a, b, esperado_suma, esperado_resta):  
        assert suma(a, b) == esperado_suma
```

```
def test_resta(self, a, b, esperado_suma, esperado_resta)
    assert resta(a, b) == esperado_resta
```

Al aplicar el decorador en la clase, cada método se ejecuta con todos los conjuntos de parámetros definidos. [1]

5. Actividad 4 – Producto cartesiano de parámetros

Objetivo

Comprender cómo se combinan dos decoradores `@pytest.mark.parametrize` para generar un producto cartesiano de casos.

Enunciado

Queremos probar todas las combinaciones posibles de x e y para una condición sencilla: $x + y \geq 2$.

Crea una prueba llamada `test_suma_minima` con:

- Un decorador `@pytest.mark.parametrize("x", [0, 1])`
- Otro decorador `@pytest.mark.parametrize("y", [1, 2])`

Haz que la prueba falle si $x + y < 2$. [1][2]

Pistas

- El orden de los decoradores importa en la generación de nombres, pero no en las combinaciones.
- Se generarán 4 combinaciones: (0,1), (1,1), (0,2), (1,2).

Solucionario (Actividad 4)

```
# archivo: test_producto_cartesiano.py
```

```
import pytest
```

```
@pytest.mark.parametrize("x", [0, 1])
@pytest.mark.parametrize("y", [1, 2])
def test_suma_minima(x, y):
    assert x + y >= 2
```

Aquí se crea el producto cartesiano de los valores de x y y, generando una prueba para cada combinación. [1][2]

6. Actividad 5 – Identificadores más legibles (opcional)

Objetivo

Dar nombres más legibles a los casos de prueba usando ids en la parametrización.

Enunciado

Supón que tienes una función `es_par(n)` que devuelve `True` si el número es par y `False` si es impar.

```
# archivo: paridad.py
```

```
def es_par(n):
    return n % 2 == 0
```

Crea una prueba parametrizada con los valores:

- 0 → True
- 1 → False
- 2 → True

Añade nombres descriptivos a los casos usando el argumento `ids` del decorador `@pytest.mark.parametrize`. [2]

Pistas

- `ids=["cero", "uno", "dos"]` es suficiente para ver nombres claros en la salida.
-

Solucionario (Actividad 5)

```
# archivo: test_paridad.py
import pytest
from paridad import es_par

@pytest.mark.parametrize("n,esperado", [
    (0, True),
    (1, False),
    (2, True),
], ids=["cero", "uno", "dos"])
def test_es_par(n, esperado):
    assert es_par(n) == esperado
```

Al ejecutar con `pytest -v`, verás nombres de caso como `test_es_par[cero]`, lo que facilita identificar qué dato ha fallado. [2]

7. Para trabajar en clase

- Recomendación: ir haciendo las actividades en orden (1 → 5).
 - No son evaluables; el objetivo es practicar el uso de `@pytest.mark.parametrize`.
 - Al final, cada estudiante debería sentirse capaz de:
 - Crear tests parametrizados básicos.
 - Combinar parámetros y usar clases.
 - Leer mejor los resultados gracias a identificadores claros.
-

[1] How to parametrize fixtures and test functions – pytest documentation. [2]

[2] Parametrizing tests – pytest documentation. [3]